

Class03d1_Iris_1_PyTorch

How to run: Double click Class03d1_Iris_1_PyTorch.ipynb to open ! Classification Task.

```
# Comment: If running in Colab, uncomment the next two lines.  
# from google.colab import drive  
# drive.mount('/content/drive')
```

Needed only in Colab for
connecting to the G-drive
for the dataset

To use ANN for the Iris Classification using the PyTorch deep learning framework.

The aim is to classify Iris flowers into three different species based on the size of their sepals and petals. This is a multi-class classification problem. The three kinds of iris are shown below: Iris setosa; Iris versicolor; Iris virginica.

Classification into one of the 3 species based on the sepals and petals dimension.



```

# Iris setosa ;      Iris versicolor ;      Iris virginica
# To import the necessary libraries
import torch
#import torchvision
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split

# numpy is a python library used for working with arrays. It also has functions working in linear algebra
# panda is an open source data analysis and manipulation tool built on top of python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

```

```
%ls
```

```

Volume in drive D is Data 2TB
Volume Serial Number is 469E-A438

```

```
Directory of D:\gpang\rebirth\Teaching\ELEC6604_2025_Sept\Course_Materials\Class05_ANN3
```

```

25/08/2025  ??04:42  <DIR>      .
25/08/2025  ??04:13  <DIR>      ..
25/08/2025  ??04:40  <DIR>      .ipynb_checkpoints
30/09/2022  ??11:09          2,206 AA-README-Class5-teaching-python.txt
20/09/2022  ??01:09       125,204 BCW_dataset.csv
05/06/2024  ??03:47       94,847 Breast-Cancer-Classification-ANN-v1.docx
05/10/2022  ??04:11       127,626 Breast-Cancer-Classification-v1.ipynb

```

```
: """The Iris data set contains the four features and one label. The four features are:  
•→sepal length;  
•→sepal width;  
•→petal length;  
•→petal width.  
The label identifies the Iris species:  
•→Iris setosa (labelled 0);  
•→Iris versicolor (labelled 1);  
•→Iris virginica. (labelled 2).  
"""
```

4 inputs

```
# Loading the Iris dataset  
dataset = pd.read_csv('iris_dataset.csv', header = None, sep = ',')  
  
dataset.columns = ["sepal length (cm)",  
                  "sepal width (cm)",  
                  "petal length (cm)",  
                  "petal width (cm)",  
                  "species"]  
  
dataset.head()
```

Readin the .csv dataset file

```
"""The Iris data set contains the four features and one label. The four features are:
•→sepal length;
•→sepal width;
•→petal length;
•→petal width.
The label identifies the Iris species:
•→Iris setosa (labelled 0);
•→Iris versicolor (labelled 1);
•→Iris virginica. (labelled 2).
"""
```

```
# Loading the Iris dataset
dataset = pd.read_csv('iris_dataset.csv', header = None, sep = ',')
```

```
dataset.columns = ["sepal length (cm)",
                  "sepal width (cm)",
                  "petal length (cm)",
                  "petal width (cm)",
                  "species"]
```

```
dataset.head()
```

Readin the .csv dataset file
if from your own filespace

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
# Loading the Iris dataset
```

```
dataset = pd.read_csv('/content/drive/MyDrive/ELEC6604_Sept2022/Class4_folder/iris_dataset.csv', header = None, sep = ',')
```

Open file with Notepad (150 rows):

If ... in Colab

 iris_dataset.csv - Notepad

File Edit Format View Help

5.1,3.5,1.4,0.2,Iris-setosa

4.9,3.0,1.4,0.2,Iris-setosa

4.7,3.2,1.3,0.2,Iris-setosa

4.6,3.1,1.5,0.2,Iris-setosa

5.0,3.6,1.4,0.2,Iris-setosa

5.4,3.9,1.7,0.4,Iris-setosa

4.6,3.4,1.4,0.3,Iris-setosa

5.0,3.4,1.5,0.2,Iris-setosa

4.4,2.9,1.4,0.2,Iris-setosa

4.9,3.1,1.5,0.1,Iris-setosa



```
# Next is to change the species data to numeric values.
```

```
mappings = {  
    "Iris-setosa": 0,  
    "Iris-versicolor": 1,  
    "Iris-virginica": 2  
}
```

```
"""a lambda function is a small anonymous function """
```

```
dataset["species"] = dataset["species"].apply(lambda x: mappings[x])
```

```
dataset.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

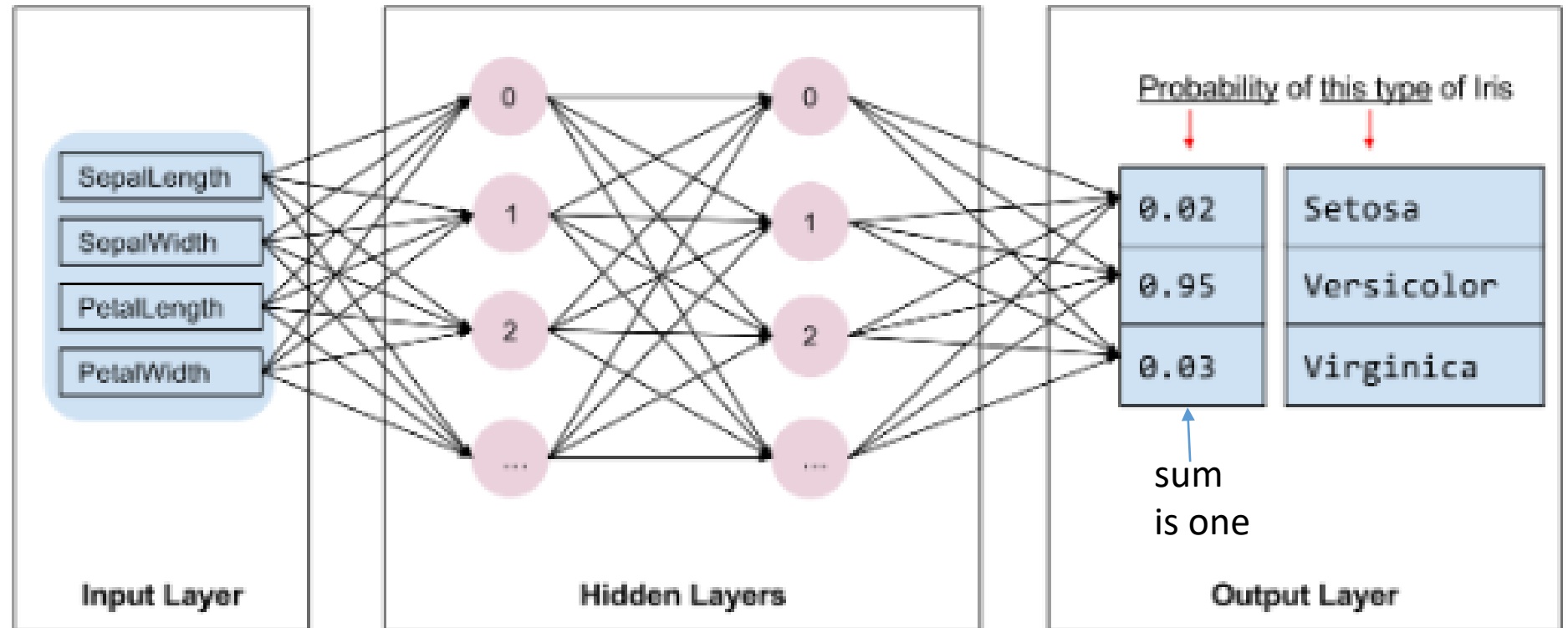
Apply Mapping: original English text is inconvenient to use

Column names

Mapping applied

The ANN architecture is converted to a three-output neural network.

The ANN architecture is converted to a three-output neural network.



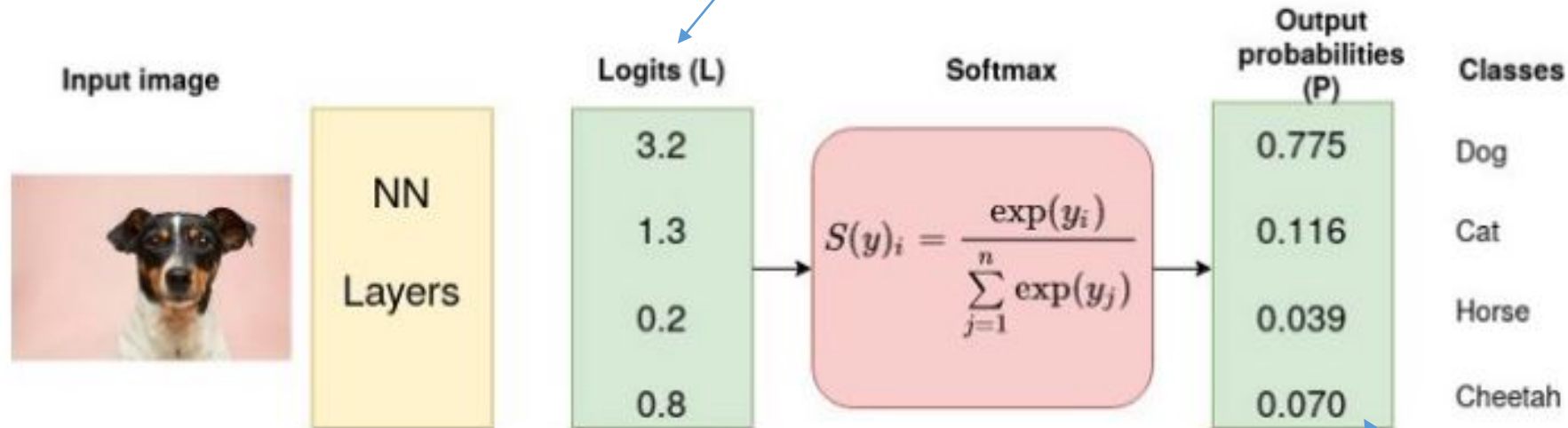
In the example above, the output indicates a 95% probability that it is an Iris Versicolor.

Loss Functions in Machine Learning for Classification

Cross Entropy

Cross entropy loss is commonly used in classification tasks both in traditional ML and deep learning.

Not good in usage



This is better.

Logits: The vector of raw (non-normalized) predictions that a classification model generates, which is ordinarily then passed to a normalization function. If the model is solving a multi-class classification problem, logits typically become an input to the softmax function. The softmax function then generates a vector of (normalized) probabilities with one value for each possible class.

Now suppose we have a training sample which is a dog, the target label would be $[1, 0, 0, 0]$, while the NN raw output is $[3.2, 1.3, 0.2, 0.8]$, the softmax probability output is $[0.775, 0.116, 0.039, 0.07]$, what would be the cross entropy loss?

$$L_{CE} = - \sum_{i=1}^n t_i \log(p_i)$$

where t_i is the binary truth label for class i , p_i is the softmax probability

Cross entropy loss guarantees a convex optimization surface and helps in gradient descent towards the minimum.

Plugging in the values, we have the loss value:

$$L_{CE} = -(1 \times \log(0.775)) + 0 \times \log(0.116) + 0 \times \log(0.039) + 0 \times \log(0.07) = 0.255$$

Note: natural log is used as common practice.

After some weights update, we have new raw outputs for the same sample, the softmax probability becomes `[0.9, 0.05, 0.03, 0.02]` , the new loss value is:

$$L_{CE} = -(1 \times \log(0.9) + 0 \times \log(0.05) + 0 \times \log(0.03) + 0 \times \log(0.02)) = 0.105$$

This new loss is lower than previous loss, indicating NN is learning.

Intuitively, we can also observe that the softmax probability is closer to the true distribution.

Another practical note, in Pytorch if one uses the `nn.CrossEntropyLoss` the input must be unnormalized raw value (aka `logits`), the target must be class index instead of one hot encoded vectors.



```
X = dataset.drop("species",axis=1).values
```

```
y = dataset["species"].values
```

Prepare inputs and outputs for training

```
# The X values are the first 4 columns of the dataset
```

```
# The y value is the last column called species
```

```
print('Dimension of X is ',X.shape)
```

```
print('Dimension of y is ',y.shape)
```

```
# Next, we load our dataset and split it into training/test sets by ratio 80/20.
```

```
# Note: 150 pieces of data: 120 for training, 30 for testing
```

```
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.20)
```

```
print('Dimension of X_train (A) is ',X_train.shape)
```

```
X_train = torch.FloatTensor(X_train)
```

```
print('Dimension of X_train (B) is ',X_train.shape)
```

```
print('Dimension of y_train is ',y_train.shape)
```

```
print('y_train: ',y_train)
```

```
print('Dimension of X_train (A) is ',X_train.shape)
X_train = torch.FloatTensor(X_train)
print('Dimension of X_train (B) is ',X_train.shape)
print('Dimension of y_train is ',y_train.shape)
print('y_train: ',y_train)
```

data type required by PyTorch



```
X_test = torch.FloatTensor(X_test)
y_train = torch.LongTensor(y_train)
y_test = torch.LongTensor(y_test)
```

Dimension of X is (150, 4)

Dimension of y is (150,)

Dimension of X_train (A) is (120, 4)

Dimension of X_train (B) is torch.Size([120, 4])

Dimension of y_train is (120,)

y_train: [0 1 1 1 1 0 0 0 2 1 2 0 1 0 2 2 0 0 1 2 0 1 2 0 1 0 0 1 1 2 1 2 1 2 1 2 0 2
2 0 0 2 2 2 1 2 1 2 0 0 0 1 1 0 2 1 2 2 2 2 2 1 0 0 2 2 2 1 0 1 1 2 1 0 1
2 0 0 1 2 0 0 0 1 1 0 2 1 0 0 1 2 2 1 1 1 0 1 1 2 1 1 1 2 2 0 0 2 2 2 0 1
2 1 0 2 1 1 1 2 1]

target output is class index in PyTorch



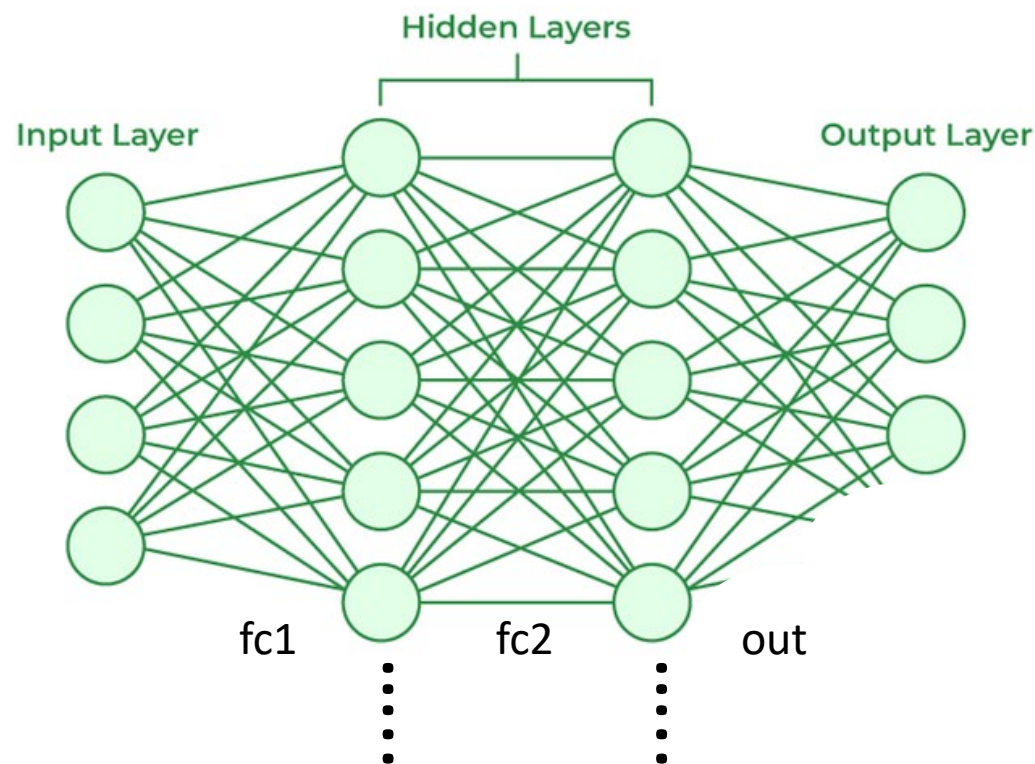
```
[ ] # Creating the ANN Model
```

```
""" Our model will have following structure:  
Fully Connected Layer1: 4 input features, 25 output features  
Fully Connected Layer2: 25 input features, 30 output features  
Output Layer:          30 input features , 3 output features  
Here we choose hidden layers' neurons as arbitrary (25 and 30 respectively).  
We will use ReLU (rectified linear unit) as activation function.  
"""
```

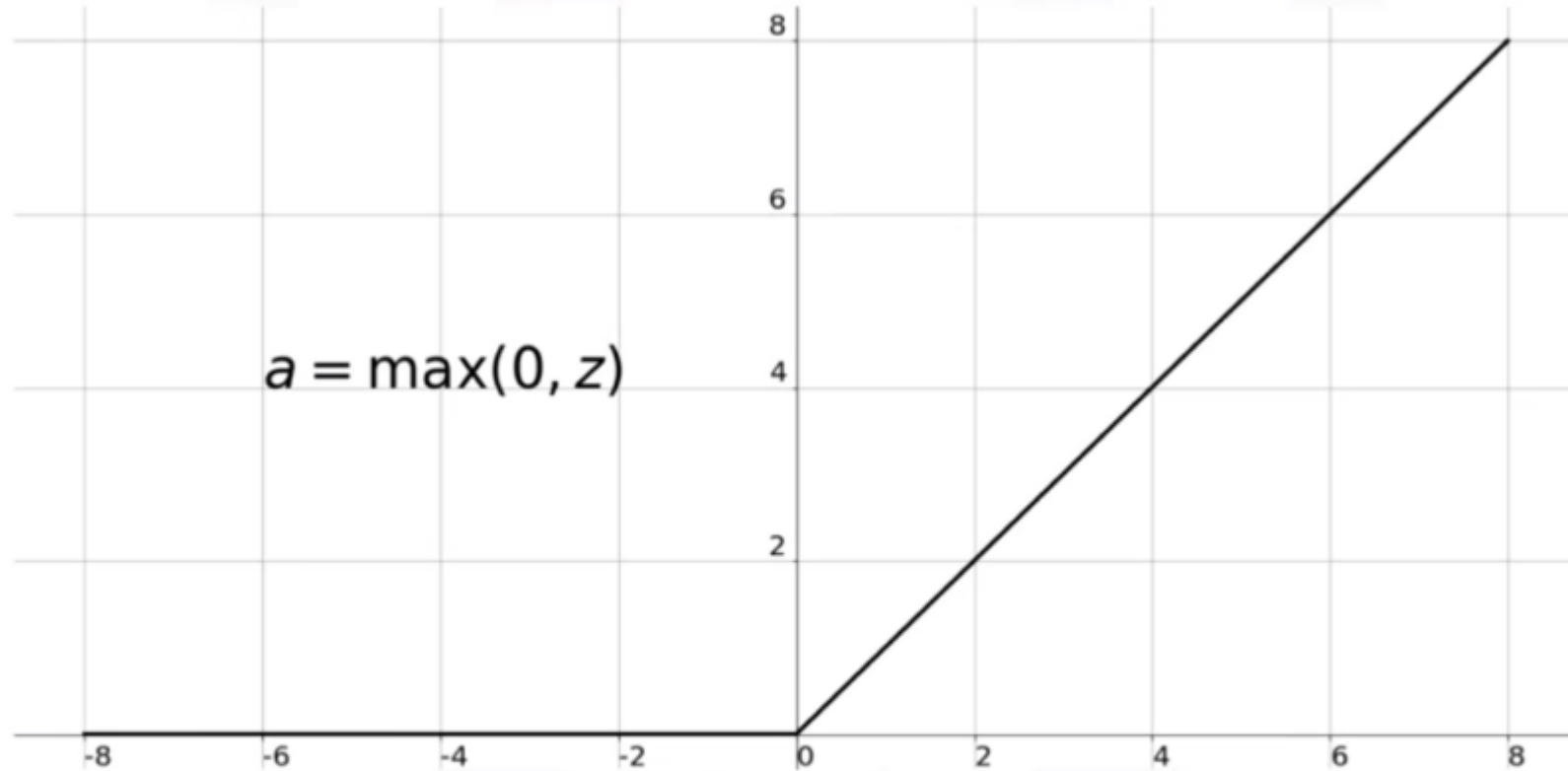
```
class Model(nn.Module):  
    def __init__(self, input_features=4, hidden_layer1=25, hidden_layer2=30, output_features=3):  
        super().__init__()  
        self.fc1 = nn.Linear(input_features, hidden_layer1)  
        self.fc2 = nn.Linear(hidden_layer1, hidden_layer2)  
        self.out = nn.Linear(hidden_layer2, output_features)  
  
    def forward(self, x):  
        x = F.relu(self.fc1(x))  
        x = F.relu(self.fc2(x))  
        x = self.out(x)  
        return x
```

```
model = Model()  
print(model)
```

```
Model(  
  (fc1): Linear(in_features=4, out_features=25, bias=True)  
  (fc2): Linear(in_features=25, out_features=30, bias=True)  
  (out): Linear(in_features=30, out_features=3, bias=True)  
,
```



ReLU Function



Setting hyper-parameters: We start with learning rate 0.01 then we can work on and see its effect on the training process. Also we will use Cross Entropy and Adam (adaptive moment estimation) optimizer.

The Adam optimization algorithm computes individual adaptive learning rates for different network parameters (weights). The Cross Entropy loss function is a popular alternative to the MSE (mean squared error) for measuring the performance of the model during training. Some researchers have shown that it can lead to faster training.

```
[ ] criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

```
# Training
epochs = 100
losses = []
losses_tmp = []
```

100 iterations

“for” loop in PyTorch

```
for i in range(epochs):
    y_pred = model.forward(X_train)
    loss = criterion(y_pred, y_train)
    losses.append(loss)
    losses_tmp.append(loss.item())
    print(f'epoch: {i:2}  loss: {loss.item():10.8f}')
    #print('  X_train.shape ', X_train.shape , 'y_pred.shape ', y_pred.shape)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Feed forward

to calculate the error

back-propagate to calculate the gradient values

gradient descend

```
epoch: 0  loss: 1.19142044
epoch: 1  loss: 1.06228757
epoch: 2  loss: 0.98648101
epoch: 3  loss: 0.93052965
epoch: 4  loss: 0.88049096
```

‘loss’ value decreasing

```
epoch: 97  loss: 0.04400074  
epoch: 98  loss: 0.04440599  
epoch: 99  loss: 0.04414474
```

```
[ ] """Note that in the main loop:  
y_pred = model.forward(X_train)
```

is forward propagation with input X_train ([120,4]) and output is y_pred ([120,3])

```
loss = criterion(y_pred, y_train)
```

is calculating the difference between the y_train ([120]) and y_pred ([120,30]).

optimizer.zero_grad() is used to clear up the gradient values in the previous epoch.

loss.backward() is used to backpropagate to calculate all the gradient values.

```
optimizer.step() is used to carry out gradient descent. """
```

```
▶ print('len(losses) = ',len(losses))
print(epochs)
x_values=np.linspace(start=1,stop=100,num=100)
print(x_values)
#print(losses)
print(losses_tmp)
```

```
len(losses) = 100
100
```

 **x-axis values**

```
[ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10. 11. 12. 13. 14.
 15. 16. 17. 18. 19. 20. 21. 22. 23. 24. 25. 26. 27. 28.
 29. 30. 31. 32. 33. 34. 35. 36. 37. 38. 39. 40. 41. 42.
 43. 44. 45. 46. 47. 48. 49. 50. 51. 52. 53. 54. 55. 56.
 57. 58. 59. 60. 61. 62. 63. 64. 65. 66. 67. 68. 69. 70.
 71. 72. 73. 74. 75. 76. 77. 78. 79. 80. 81. 82. 83. 84.
 85. 86. 87. 88. 89. 90. 91. 92. 93. 94. 95. 96. 97. 98.
 99. 100.]
```

 **losses_tmp**

```
[1.1914204359054565, 1.0622875690460205, 0.9864810109138489, 0.9305296540260315, 0.880
```

```
[ ] # To visualize the training results
plt.plot(x_values, losses_tmp)
plt.ylabel('Loss')
plt.xlabel('epoch');
plt.show() # to show the Figure
```

```
#Validating and testing the model
```

```
# X_test.shape is [30,4]
```

```
preds = []
```

```
with torch.no_grad():
```

```
    for val in X_test:
```

```
        y_hat = model.forward(val)
```

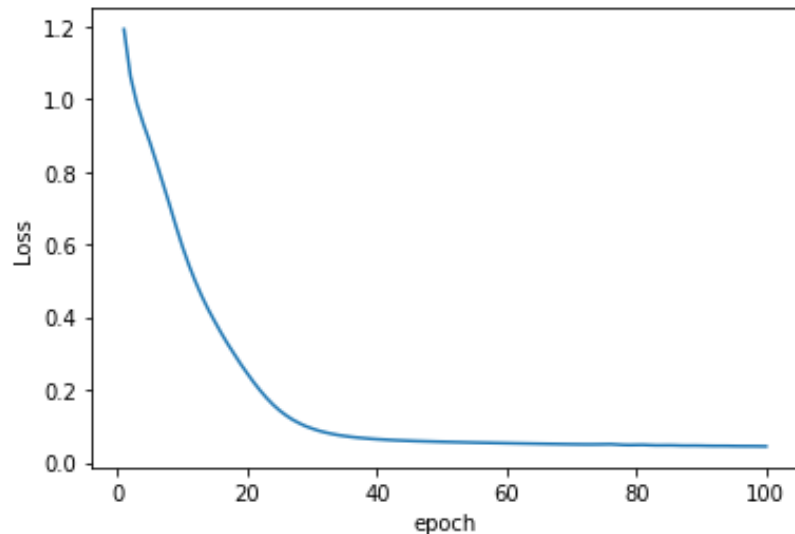
```
        preds.append(y_hat.argmax().item())
```

```
print('preds          =',preds)
```

```
print('y_test = ',y_test)
```

X_test : test input from test set

y_test : desired output from test set



y_test : desired output from test set

```
preds          = [2, 0, 2, 1, 0, 1, 2, 0, 0, 1, 0, 0, 2, 2, 1, 0, 0, 2, 0, 0, 1, 1, 0, 0, 2, 0, 2, 0, 1, 1]
y_test = tensor([2, 0, 2, 1, 0, 1, 2, 0, 0, 2, 0, 0, 2, 2, 1, 0, 0, 2, 0, 0, 1, 1, 0, 0, 2, 0, 2, 0, 1, 1])
                2, 0, 2, 0, 1, 1])
```

EXPLANATIONS:
X_test is [30,4] y_test is [30,1]
input (val) is [4] i.e. four inputs
output (y_hat) is [3] i.e. three outputs
y_hat.argmax().item() is to get the argument which is the largest.
Example: we use the last item in the X_test, and provide as input to the trained ANN.
The three outputs are given as y_ANN.
The last output item (11.6706) is the largest and the answer is index 2,
which is the same as y_test[29] (Iris-virginica).
"""

' EXPLANATIONS:\nX_test is [30,4] y_test is [30,1]\ninput (val) is [4] i.e. four :
three outputs\nny_hat.argmax().item() is to get the argument which is the largest.\nExar
_test, and provide as input to the trained ANN. \n\nThe three outputs are given as y_ANN.
is the largest and the answer is index 2, \n\nwhich is the same as y_test[29] (Iris-virg

Example based on the last item in X_test:

```
>>> Xtest30=X_test[29]
>>> Xtest30
tensor([6.7000, 3.1000, 5.6000, 2.4000])
>>> y_ANN = model.forward(Xtest30)
>>> y_ANN
tensor([-11.4423,  2.0958, 11.6706], grad_fn=<AddBackward0>)
>>> y_ANN.argmax().item()
2
>>> y_test[29]
tensor(2)
```

4 inputs

3 outputs

argmax():get the argument which is the largest

```
[ ] """ Testing the model with unseen data
unknown_iris = torch.tensor([4.0,3.3,1.7,0.5])

with torch.no_grad():
    print(model(unknown_iris))
    print()
    print(labels[model(unknown_iris).argmax()])
```

argmax():get the argument which is the largest, which is 0, Iris-setosa
tensor([12.0534, 7.1283, -17.9192])
12.0534 is the largest value, 1st class is the answer
Iris-setosa

""" A summary of this demo:

- Import the relevant libraries and Load the dataset
- Create a model
- Training the model
- Testing the model with unseen data